



Chapter 23 – Project planning

Topics covered



- ✧ Software pricing
- ✧ Plan-driven development
- ✧ Project scheduling
- ✧ Agile planning
- ✧ Estimation techniques
- ✧ COCOMO cost modeling

Project planning



- ✧ Project planning involves breaking down the work into parts and assign these to project team members, anticipate problems that might arise and prepare tentative solutions to those problems.
- ✧ The project plan, which is created at the start of a project, is used to communicate how the work will be done to the project team and customers, and to help assess progress on the project.

Planning stages



- ✧ At the proposal stage, when you are bidding for a contract to develop or provide a software system.
- ✧ During the project startup phase, when you have to plan who will work on the project, how the project will be broken down into increments, how resources will be allocated across your company, etc.
- ✧ Periodically throughout the project, when you modify your plan in the light of experience gained and information from monitoring the progress of the work.

Proposal planning



- ✧ Planning may be necessary with only outline software requirements.
- ✧ The aim of planning at this stage is to provide information that will be used in setting a price for the system to customers.
- ✧ Project pricing involves estimating how much the software will cost to develop, taking factors such as staff costs, hardware costs, software costs, etc. into account

Project startup planning



- ✧ At this stage, you know more about the system requirements but do not have design or implementation information
- ✧ Create a plan with enough detail to make decisions about the project budget and staffing.
 - This plan is the basis for project resource allocation
- ✧ The startup plan should also define project monitoring mechanisms
- ✧ A startup plan is still needed for agile development to allow resources to be allocated to the project

Development planning



- ✧ The project plan should be regularly amended as the project progresses and you know more about the software and its development
- ✧ The project schedule, cost-estimate and risks have to be regularly revised



Software pricing

Software pricing



- ✧ Estimates are made to discover the cost, to the developer, of producing a software system.
 - You take into account, hardware, software, travel, training and effort costs.
- ✧ There is not a simple relationship between the development cost and the price charged to the customer.
- ✧ Broader organisational, economic, political and business considerations influence the price charged.

Factors affecting software pricing



Factor	Description
Contractual terms	A customer may be willing to allow the developer to retain ownership of the source code and reuse it in other projects. The price charged may then be less than if the software source code is handed over to the customer.
Cost estimate uncertainty	If an organization is unsure of its cost estimate, it may increase its price by a contingency over and above its normal profit.
Financial health	Developers in financial difficulty may lower their price to gain a contract. It is better to make a smaller than normal profit or break even than to go out of business. Cash flow is more important than profit in difficult economic times.

Factors affecting software pricing



Factor	Description
Market opportunity	A development organization may quote a low price because it wishes to move into a new segment of the software market. Accepting a low profit on one project may give the organization the opportunity to make a greater profit later. The experience gained may also help it develop new products.
Requirements volatility	If the requirements are likely to change, an organization may lower its price to win a contract. After the contract is awarded, high prices can be charged for changes to the requirements.

Pricing strategies



✧ Under pricing

- A company may underprice a system in order to gain a contract that allows them to retain staff for future opportunities
- A company may underprice a system to gain access to a new market area

✧ Increased pricing

- The price may be increased when a buyer wishes a fixed-price contract and so the seller increases the price to allow for unexpected risks

Pricing to win



- ✧ The software is priced according to what the software developer believes the buyer is willing to pay
- ✧ If this is less than the development costs, the software functionality may be reduced accordingly with a view to extra functionality being added in a later release
- ✧ Additional costs may be added as the requirements change and these may be priced at a higher level to make up the shortfall in the original price

Ethical Implications of Quoting Low Prices for Software Contracts with Ambiguous Requirements



- ✧ No, it is generally unethical for a company to quote a low price for a software contract knowing the requirements are ambiguous and planning to charge high prices for subsequent changes. This practice misleads customers, undermines trust, and violates professional integrity. Instead, companies should provide realistic estimates that reflect the true scope of the project, clearly communicate any uncertainties, and work collaboratively with customers to clarify requirements and anticipate potential costs. This approach fosters transparency, fairness, and a trust-based relationship, which ultimately benefits both the customer and the company. Ethical practices ensure long-term customer satisfaction and a positive business reputation.



Plan-driven development

Plan-driven development



- ✧ Plan-driven or plan-based development is an approach to software engineering where the development process is planned in detail.
 - Plan-driven development is based on engineering project management techniques and is the 'traditional' way of managing large software development projects.
- ✧ A project plan is created that records the work to be done, who will do it, the development schedule and the work products.
- ✧ Managers use the plan to support project decision making and as a way of measuring progress.

Plan-driven development – pros and cons



- ✧ The arguments in favor of a plan-driven approach are that early planning allows organizational issues (availability of staff, other projects, etc.) to be closely taken into account, and that potential problems and dependencies are discovered before the project starts, rather than once the project is underway.
- ✧ The principal argument against plan-driven development is that many early decisions have to be revised because of changes to the environment in which the software is to be developed and used.

Project plans



✧ In a plan-driven development project, a project plan sets out the resources available to the project, the work breakdown and a schedule for carrying out the work.

✧ Plan sections

- Introduction
- Project organization
- Risk analysis
- Hardware and software resource requirements
- Work breakdown
- Project schedule
- Monitoring and reporting mechanisms

Project plan supplements



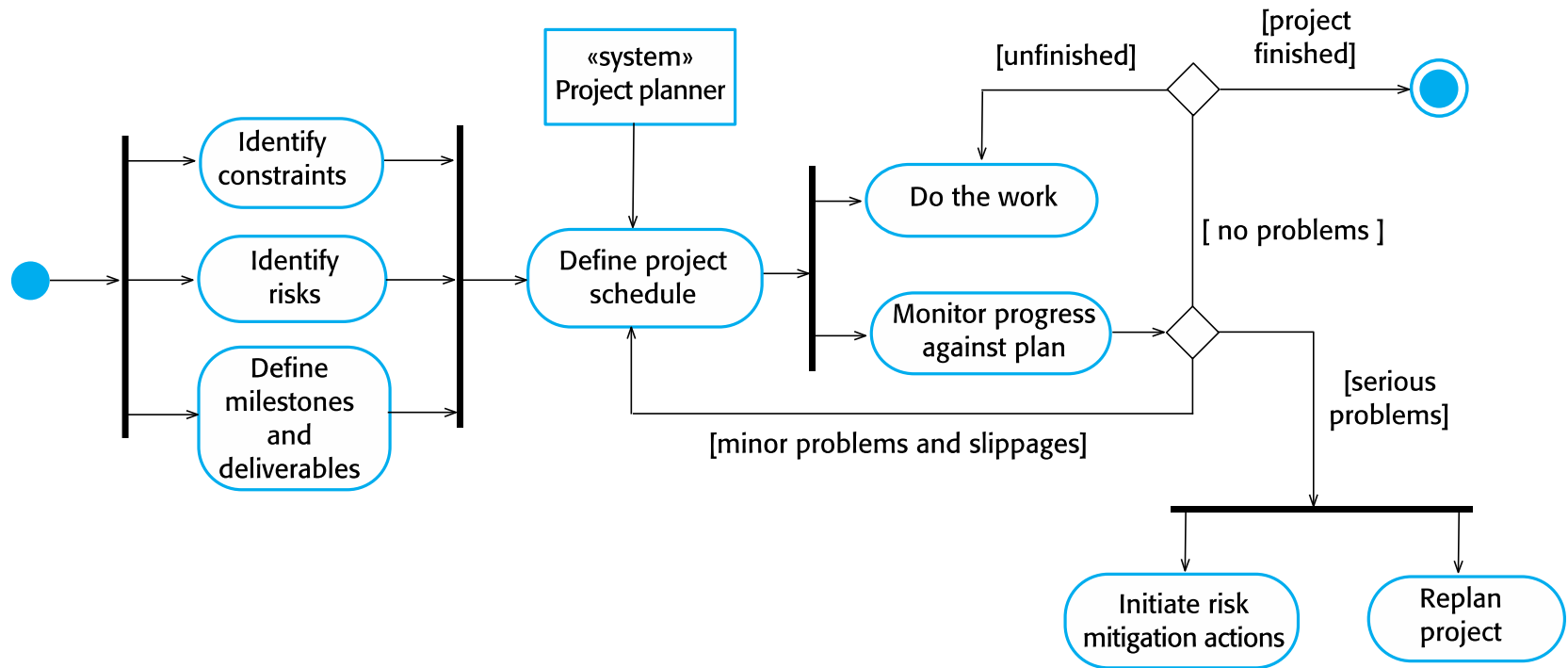
Plan	Description
Configuration management plan	Describes the configuration management procedures and structures to be used.
Deployment plan	Describes how the software and associated hardware (if required) will be deployed in the customer's environment. This should include a plan for migrating data from existing systems.
Maintenance plan	Predicts the maintenance requirements, costs, and effort.
Quality plan	Describes the quality procedures and standards that will be used in a project.
Validation plan	Describes the approach, resources, and schedule used for system validation.

The planning process



- ✧ Project planning is an iterative process that starts when you create an initial project plan during the project startup phase.
- ✧ Plan changes are inevitable.
 - As more information about the system and the project team becomes available during the project, you should regularly revise the plan to reflect requirements, schedule and risk changes.
 - Changing business goals also leads to changes in project plans. As business goals change, this could affect all projects, which may then have to be re-planned.

The project planning process



Planning assumptions



- ✧ You should make realistic rather than optimistic assumptions when you are defining a project plan.
- ✧ Problems of some description always arise during a project, and these lead to project delays.
- ✧ Your initial assumptions and scheduling should therefore take unexpected problems into account.
- ✧ You should include contingency in your plan so that if things go wrong, then your delivery schedule is not seriously disrupted.

Risk mitigation



- ✧ If there are serious problems with the development work that are likely to lead to significant delays, you need to initiate risk mitigation actions to reduce the risks of project failure.
- ✧ In conjunction with these actions, you also have to re-plan the project.
- ✧ This may involve renegotiating the project constraints and deliverables with the customer. A new schedule of when work should be completed also has to be established and agreed with the customer.



Project scheduling

Project scheduling



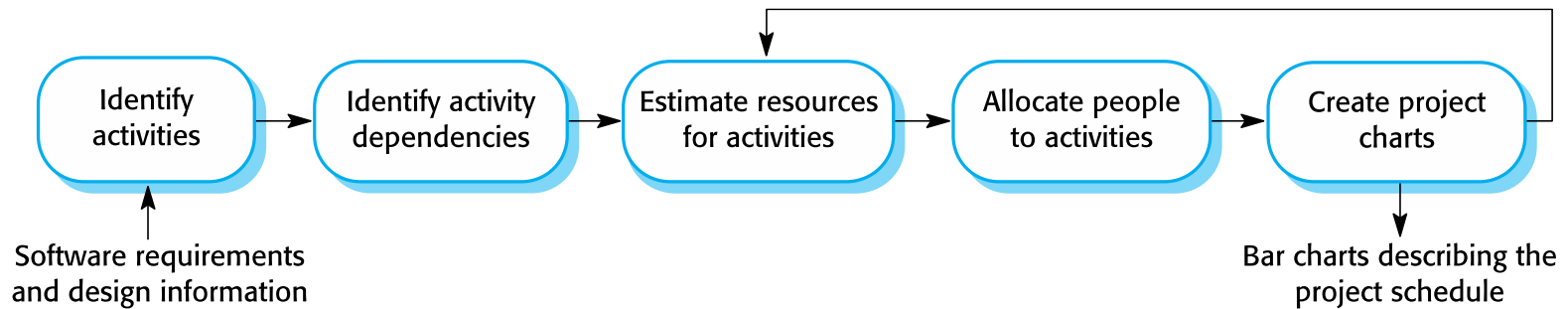
- ✧ Project scheduling is the process of deciding how the work in a project will be organized as separate tasks, and when and how these tasks will be executed.
- ✧ You estimate the calendar time needed to complete each task, the effort required and who will work on the tasks that have been identified.
- ✧ You also have to estimate the resources needed to complete each task, such as the disk space required on a server, the time required on specialized hardware, such as a simulator, and what the travel budget will be.

Project scheduling activities



- ✧ Split project into tasks and estimate time and resources required to complete each task.
- ✧ Organize tasks concurrently to make optimal use of workforce.
- ✧ Minimize task dependencies to avoid delays caused by one task waiting for another to complete.
- ✧ Dependent on project managers intuition and experience.

The project scheduling process



Scheduling problems



- ✧ Estimating the difficulty of problems and hence the cost of developing a solution is hard.
- ✧ Productivity is not proportional to the number of people working on a task.
- ✧ Adding people to a late project makes it later because of communication overheads.
- ✧ The unexpected always happens. Always allow contingency in planning.

Schedule presentation



- ✧ Graphical notations are normally used to illustrate the project schedule.
- ✧ These show the project breakdown into tasks. Tasks should not be too small. They should take about a week or two.
- ✧ Calendar-based
 - Bar charts are the most commonly used representation for project schedules. They show the schedule as activities or resources against time.
- ✧ Activity networks
 - Show task dependencies

Project activities



- ✧ Project activities (tasks) are the basic planning element.
Each activity has:
- a duration in calendar days or months,
 - an effort estimate, which shows the number of person-days or person-months to complete the work,
 - a deadline by which the activity should be complete,
 - a defined end-point, which might be a document, the holding of a review meeting, the successful execution of all tests, etc.

Milestones and deliverables



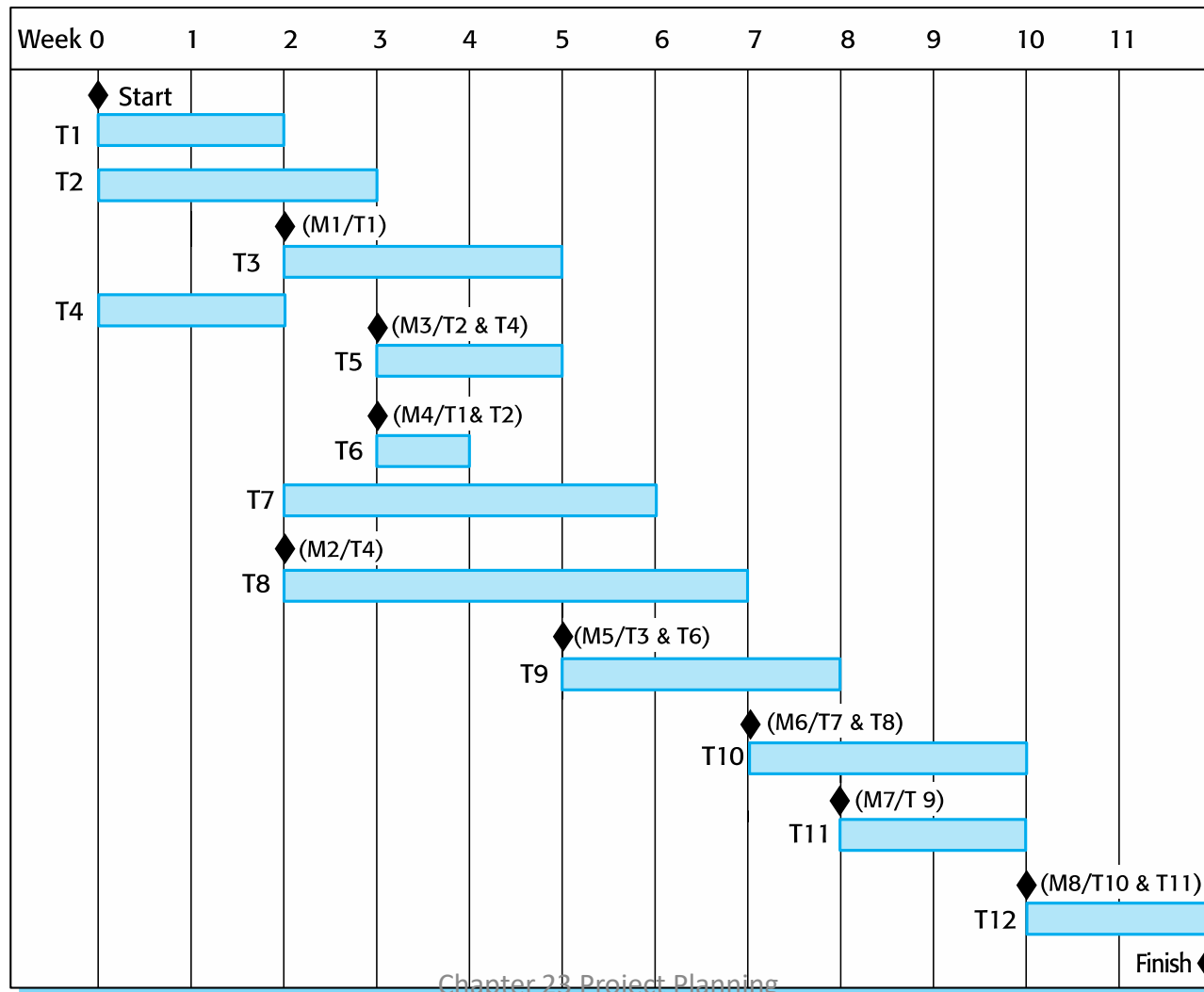
- ✧ Milestones are points in the schedule against which you can assess progress, for example, the handover of the system for testing.
- ✧ Deliverables are work products that are delivered to the customer, e.g. a requirements document for the system.

Tasks, durations, and dependencies

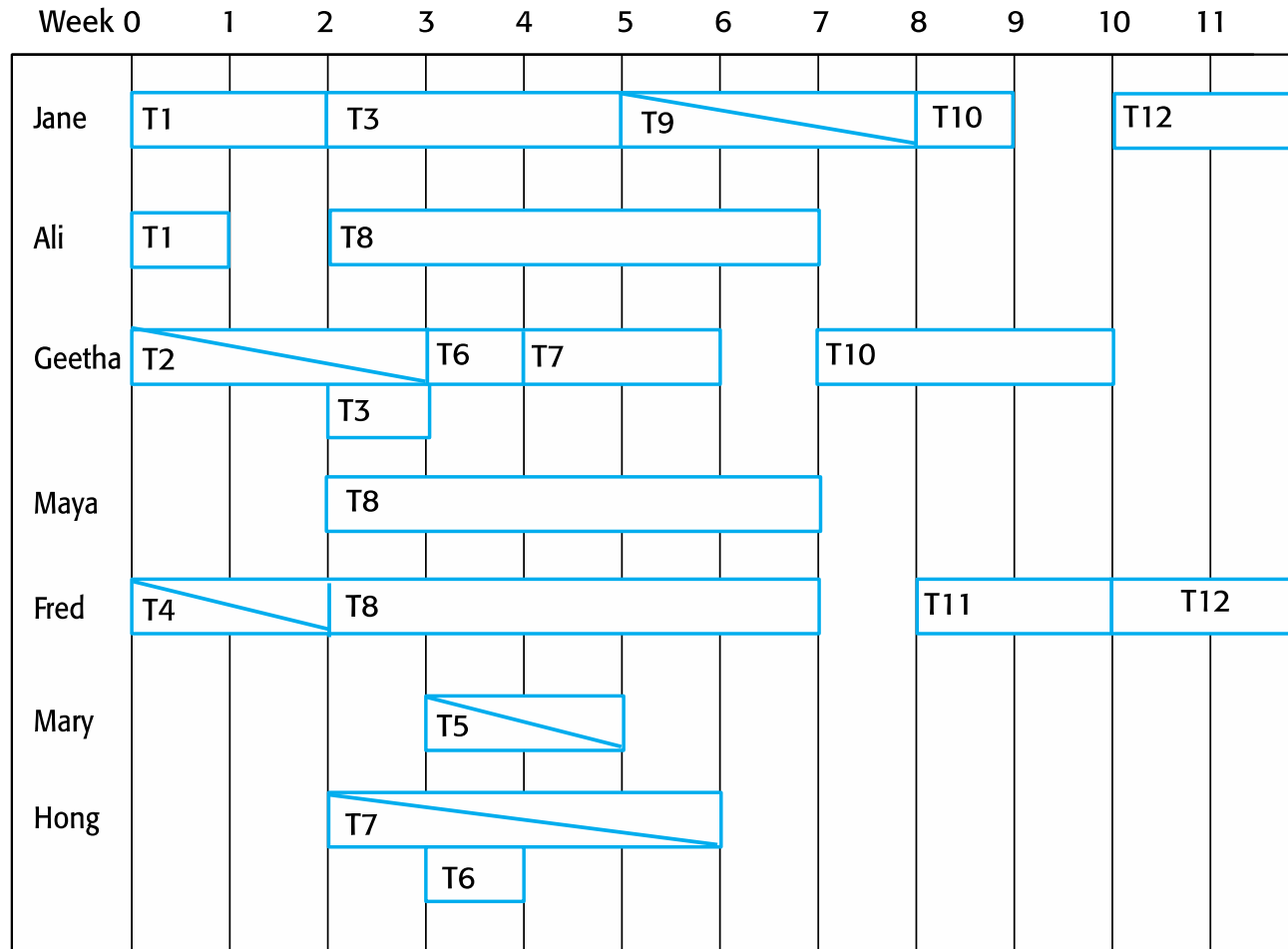


Task	Effort (person-days)	Duration (days)	Dependencies
T1	15	10	
T2	8	15	
T3	20	15	T1 (M1)
T4	5	10	
T5	5	10	T2, T4 (M3)
T6	10	5	T1, T2 (M4)
T7	25	20	T1 (M1)
T8	75	25	T4 (M2)
T9	10	15	T3, T6 (M5)
T10	20	15	T7, T8 (M6)
T11	10	10	T9 (M7)
T12	20	10	T10, T11 (M8)

Activity bar chart



Staff allocation chart





Agile planning

Agile planning



- ✧ Agile methods of software development are iterative approaches where the software is developed and delivered to customers in increments.
- ✧ Unlike plan-driven approaches, the functionality of these increments is not planned in advance but is decided during the development.
 - The decision on what to include in an increment depends on progress and on the customer's priorities.
- ✧ The customer's priorities and requirements change so it makes sense to have a flexible plan that can accommodate these changes.

Agile planning stages



- ✧ Release planning, which looks ahead for several months and decides on the features that should be included in a release of a system.
- ✧ Iteration planning, which has a shorter term outlook, and focuses on planning the next increment of a system. This is typically 2-4 weeks of work for the team.

Approaches to agile planning



✧ Planning in Scrum

- Covered in Chapter 3

✧ Based on managing a project backlog (things to be done) with daily reviews of progress and problems

✧ The planning game

- Developed originally as part of Extreme Programming (XP)
- Dependent on user stories as a measure of progress in the project

Story-based planning



- ✧ The planning game is based on user stories that reflect the features that should be included in the system.
- ✧ The project team read and discuss the stories and rank them in order of the amount of time they think it will take to implement the story.
- ✧ Stories are assigned 'effort points' reflecting their size and difficulty of implementation
- ✧ The number of effort points implemented per day is measured giving an estimate of the team's 'velocity'
- ✧ This allows the total effort required to implement the system to be estimated

The planning game



Release and iteration planning



- ✧ Release planning involves selecting and refining the stories that will reflect the features to be implemented in a release of a system and the order in which the stories should be implemented.
- ✧ Stories to be implemented in each iteration are chosen, with the number of stories reflecting the time to deliver an iteration (usually 2 or 3 weeks).
- ✧ The team's velocity is used to guide the choice of stories so that they can be delivered within an iteration.

Task allocation



- ✧ During the task planning stage, the developers break down stories into development tasks.
 - A development task should take 4–16 hours.
 - All of the tasks that must be completed to implement all of the stories in that iteration are listed.
 - The individual developers then sign up for the specific tasks that they will implement.
- ✧ Benefits of this approach:
 - The whole team gets an overview of the tasks to be completed in an iteration.
 - Developers have a sense of ownership in these tasks and this is likely to motivate them to complete the task.

Software delivery



- ✧ A software increment is always delivered at the end of each project iteration.
- ✧ If the features to be included in the increment cannot be completed in the time allowed, the scope of the work is reduced.
- ✧ The delivery schedule is never extended.

Agile planning difficulties



- ✧ Agile planning is reliant on customer involvement and availability.
- ✧ This can be difficult to arrange, as customer representatives sometimes have to prioritize other work and are not available for the planning game.
- ✧ Furthermore, some customers may be more familiar with traditional project plans and may find it difficult to engage in an agile planning process.

Agile planning applicability



- ✧ Agile planning works well with small, stable development teams that can get together and discuss the stories to be implemented.
- ✧ However, where teams are large and/or geographically distributed, or when team membership changes frequently, it is practically impossible for everyone to be involved in the collaborative planning that is essential for agile project management.



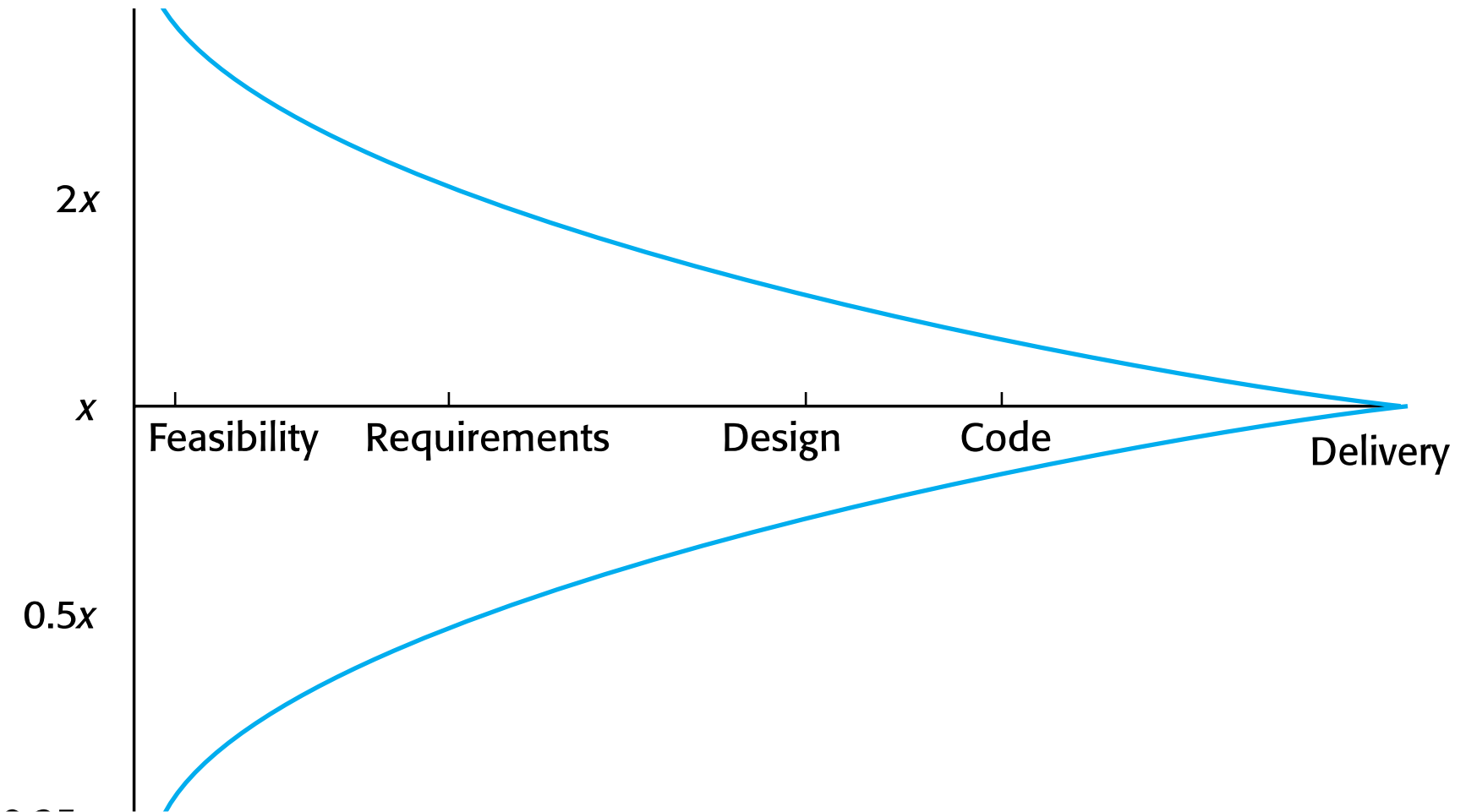
Estimation techniques

Estimation techniques



- ✧ Organizations need to make software effort and cost estimates. There are two types of technique that can be used to do this:
 - *Experience-based techniques* The estimate of future effort requirements is based on the manager's experience of past projects and the application domain. Essentially, the manager makes an informed judgment of what the effort requirements are likely to be.
 - *Algorithmic cost modeling* In this approach, a formulaic approach is used to compute the project effort based on estimates of product attributes, such as size, and process characteristics, such as experience of staff involved.

Estimate uncertainty



Experience-based approaches



- ✧ Experience-based techniques rely on judgments based on experience of past projects and the effort expended in these projects on software development activities.
- ✧ Typically, you identify the deliverables to be produced in a project and the different software components or systems that are to be developed.
- ✧ You document these in a spreadsheet, estimate them individually and compute the total effort required.
- ✧ It usually helps to get a group of people involved in the effort estimation and to ask each member of the group to explain their estimate.

Problem with experience-based approaches



- ✧ The difficulty with experience-based techniques is that a new software project may not have much in common with previous projects.
- ✧ Software development changes very quickly and a project will often use unfamiliar techniques such as web services, application system configuration or HTML5.
- ✧ If you have not worked with these techniques, your previous experience may not help you to estimate the effort required, making it more difficult to produce accurate costs and schedule estimates.

Algorithmic cost modelling



- ✧ Cost is estimated as a mathematical function of product, project and process attributes whose values are estimated by project managers:
 - $\text{Effort} = A \cdot \text{Size}^B \cdot M$
 - A is an organisation-dependent constant, B reflects the disproportionate effort for large projects and M is a multiplier reflecting product, process and people attributes.
- ✧ The most commonly used product attribute for cost estimation is code size.
- ✧ Most models are similar but they use different values for A, B and M.

Estimation accuracy



- ✧ The size of a software system can only be known accurately when it is finished.
- ✧ Several factors influence the final size
 - Use of reused systems and components;
 - Programming language;
 - Distribution of system.
- ✧ As the development process progresses then the size estimate becomes more accurate.
- ✧ The estimates of the factors contributing to B and M are subjective and vary according to the judgment of the estimator.

Effectiveness of algorithmic models



- ✧ Algorithmic cost models are a systematic way to estimate the effort required to develop a system. However, these models are complex and difficult to use.
- ✧ There are many attributes and considerable scope for uncertainty in estimating their values.
- ✧ This complexity means that the practical application of algorithmic cost modeling has been limited to a relatively small number of large companies, mostly working in defense and aerospace systems engineering.

COCOMO cost modeling

COCOMO cost modeling



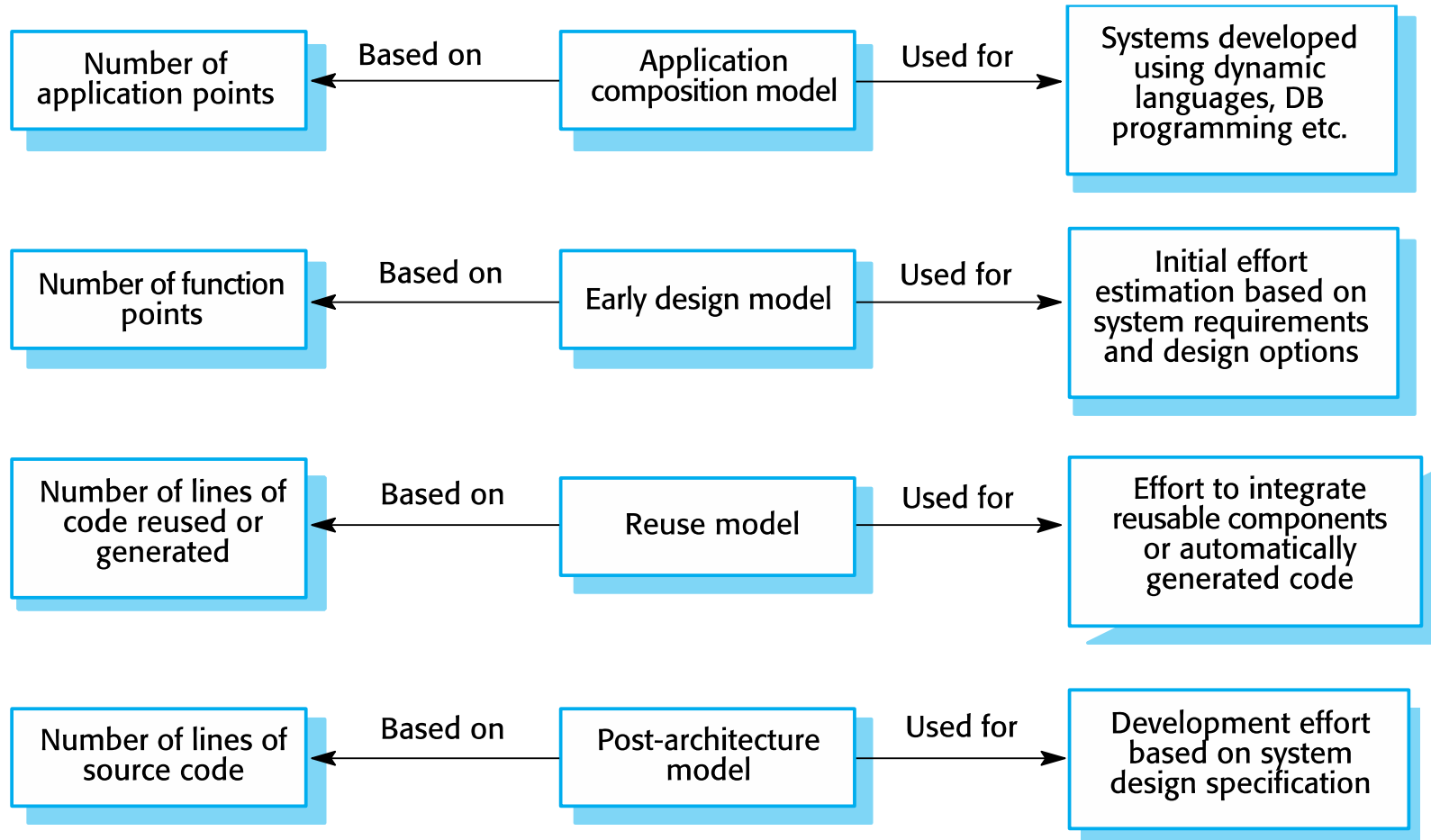
- ✧ An empirical model based on project experience.
- ✧ Well-documented, 'independent' model which is not tied to a specific software vendor.
- ✧ Long history from initial version published in 1981 (COCOMO-81) through various instantiations to COCOMO 2.
- ✧ COCOMO 2 takes into account different approaches to software development, reuse, etc.

COCOMO 2 models



- ✧ COCOMO 2 incorporates a range of sub-models that produce increasingly detailed software estimates.
- ✧ The sub-models in COCOMO 2 are:
 - **Application composition model**. Used when software is composed from existing parts.
 - **Early design model**. Used when requirements are available but design has not yet started.
 - **Reuse model**. Used to compute the effort of integrating reusable components.
 - **Post-architecture model**. Used once the system architecture has been designed and more information about the system is available.

COCOMO estimation models



Application composition model



- ✧ Supports prototyping projects and projects where there is extensive reuse.
- ✧ Based on standard estimates of developer productivity in application (object) points/month.
- ✧ Takes software tool use into account.
- ✧ Formula is
 - $PM = (NAP \cdot (1 - \%reuse/100)) / PROD$
 - PM is the effort in person-months, NAP is the number of application points and PROD is the productivity.

Application-point productivity



Developer's experience and capability	Very low	Low	Nominal	High	Very high
ICASE maturity and capability	Very low	Low	Nominal	High	Very high
PROD (NAP/month)	4	7	13	25	50

Early design model



- ✧ Estimates can be made after the requirements have been agreed.
- ✧ Based on a standard formula for algorithmic models
- ✧ $PM = A \cdot Size^B \cdot M$ where
 - $M = PERS \cdot RCPX \cdot RUSE \cdot PDIF \cdot PREX \cdot FCIL \cdot SCED$;
 - $A = 2.94$ in initial calibration,
 - Size in KLOC,
 - B varies from 1.1 to 1.24 depending on novelty of the project, development flexibility, risk management approaches and the process maturity.

Multipliers



- ✧ Multipliers reflect the capability of the developers, the non-functional requirements, the familiarity with the development platform, etc.
 - RCPX - product reliability and complexity;
 - RUSE - the reuse required;
 - PDIF - platform difficulty;
 - PREX - personnel experience;
 - PERS - personnel capability;
 - SCED - required schedule;
 - FCIL - the team support facilities.

The reuse model



- ✧ Takes into account black-box code that is reused without change and code that has to be adapted to integrate it with new code.
- ✧ There are two versions:
 - Black-box reuse where code is not modified. An effort estimate (PM) is computed.
 - White-box reuse where code is modified. A size estimate equivalent to the number of lines of new source code is computed. This then adjusts the size estimate for new code.

Reuse model estimates 1



✧ For generated code:

✧ $PM = (ASLOC * AT/100)/ATPROD$

- ASLOC is the number of lines of generated code
- AT is the percentage of code automatically generated.
- ATPROD is the productivity of engineers in integrating this code.

Reuse model estimates 2



- ✧ When code has to be understood and integrated:
- ✧ $ESLOC = ASLOC * (1 - AT/100) * AAM$.
 - ASLOC and AT as before.
 - AAM is the adaptation adjustment multiplier computed from the costs of changing the reused code, the costs of understanding how to integrate the code and the costs of reuse decision making.

Post-architecture level



- ✧ Uses the same formula as the early design model but with 17 rather than 7 associated multipliers.
- ✧ The code size is estimated as:
 - Number of lines of new code to be developed;
 - Estimate of equivalent number of lines of new code computed using the reuse model;
 - An estimate of the number of lines of code that have to be modified according to requirements changes.

The exponent term



- ✧ This depends on 5 scale factors (see next slide). Their sum/100 is added to 1.01
- ✧ A company takes on a project in a new domain. The client has not defined the process to be used and has not allowed time for risk analysis. The company has a CMM level 2 rating.
 - Precedenteness - new project (4)
 - Development flexibility - no client involvement - Very high (1)
 - Architecture/risk resolution - No risk analysis - V. Low .(5)
 - Team cohesion - new team - nominal (3)
 - Process maturity - some control - nominal (3)
- ✧ Scale factor is therefore 1.17.

Scale factors used in the exponent computation in the post-architecture model



Scale factor	Explanation
Architecture/risk resolution	Reflects the extent of risk analysis carried out. Very low means little analysis; extra-high means a complete and thorough risk analysis.
Development flexibility	Reflects the degree of flexibility in the development process. Very low means a prescribed process is used; extra-high means that the client sets only general goals.
Precedentedness	Reflects the previous experience of the organization with this type of project. Very low means no previous experience; extra-high means that the organization is completely familiar with this application domain.
Process maturity	Reflects the process maturity of the organization. The computation of this value depends on the CMM Maturity Questionnaire, but an estimate can be achieved by subtracting the CMM process maturity level from 5.
Team cohesion	Reflects how well the development team knows each other and work together. Very low means very difficult interactions; extra-high means an integrated and effective team with no communication problems.

Multipliers



✧ Product attributes

- Concerned with required characteristics of the software product being developed.

✧ Computer attributes

- Constraints imposed on the software by the hardware platform.

✧ Personnel attributes

- Multipliers that take the experience and capabilities of the people working on the project into account.

✧ Project attributes

- Concerned with the particular characteristics of the software development project.

The effect of cost drivers on effort estimates



Exponent value	1.17
System size (including factors for reuse and requirements volatility)	128,000 DSI
Initial COCOMO estimate without cost drivers	730 person-months
Reliability	Very high, multiplier = 1.39
Complexity	Very high, multiplier = 1.3
Memory constraint	High, multiplier = 1.21
Tool use	Low, multiplier = 1.12
Schedule	Accelerated, multiplier = 1.29
Adjusted COCOMO estimate	2,306 person-months

The effect of cost drivers on effort estimates



Exponent value	1.17
Reliability	Very low, multiplier = 0.75
Complexity	Very low, multiplier = 0.75
Memory constraint	None, multiplier = 1
Tool use	Very high, multiplier = 0.72
Schedule	Normal, multiplier = 1
Adjusted COCOMO estimate	295 person-months

Project duration and staffing



- ✧ As well as effort estimation, managers must estimate the calendar time required to complete a project and when staff will be required.
- ✧ Calendar time can be estimated using a COCOMO 2 formula
 - $TDEV = 3 \cdot (PM)^{(0.33+0.2 \cdot (B-1.01))}$
 - PM is the effort computation and B is the exponent computed as discussed above (B is 1 for the early prototyping model). This computation predicts the nominal schedule for the project.
- ✧ The time required is independent of the number of people working on the project.

Staffing requirements



- ✧ Staff required can't be computed by dividing the development time by the required schedule.
- ✧ The number of people working on a project varies depending on the phase of the project.
- ✧ The more people who work on the project, the more total effort is usually required.
- ✧ A very rapid build-up of people often correlates with schedule slippage.

Key points



- ✧ The price charged for a system does not just depend on its estimated development costs and the profit required by the development company. Organizational factors may mean that the price is increased to compensate for increased risk or decreased to gain competitive advantage.
- ✧ Software is often priced to gain a contract and the functionality of the system is then adjusted to meet the estimated price.
- ✧ Plan-driven development is organized around a complete project plan that defines the project activities, the planned effort, the activity schedule and who is responsible for each activity.

Key points



- ✧ Project scheduling involves the creation of various graphical representations of part of the project plan. Bar charts, which show the activity duration and staffing timelines, are the most commonly used schedule representations.
- ✧ A project milestone is a predictable outcome of an activity or set of activities. At each milestone, a formal report of progress should be presented to management. A deliverable is a work product that is delivered to the project customer.
- ✧ The agile planning game involves the whole team in project planning. The plan is developed incrementally and, if problems arise, it is adjusted so that software functionality is reduced instead of delaying the delivery of an increment.

Key points



- ✧ Estimation techniques for software may be experience-based, where managers judge the effort required, or algorithmic, where the effort required is computed from other estimated project parameters.
- ✧ The COCOMO II costing model is a mature algorithmic cost model that takes project, product, hardware and personnel attributes into account when formulating a cost estimate.